COMP1610 Coursework Report

Programming Enterprise Components

Bank App Project

Jack Jibb

001408490

Other Group Members:

Syed Ali Irtaza – 001414068

Anastasiia Pyslar – 001412071

Devanshi Pandya – 001426270

Poorva Motaval – 001449334

Submission Date: 24 March 2025

Contents

1 Design Documentation (Group Work) 3
1.1 Entity-Relationship Diagram . . . 3
1.2 Architecture Diagram . . . 4

2 Screenshots (Group Work) 4
2.0.1 Authentication & Session Management . . . 5
2.0.2 Student Management (CRUD) . . . 5
2.0.3 Account Management (CRUD) . . . 6
2.0.4 Transactions (Deposit, Withdraw, Transfer) . . . 6
2.0.5 Front-End Features . . . 6
2.0.6 RESTful API (JSON) . . . 6

3 Evaluation (Group Work) 7

4 Research (Individual) 7

5 Individual Reflection (Individual) 7

6 Appendix 7
6.1 WildFly 33 Server Configuration . . . 7
6.1.1 Subsystems and Extensions . . . 7
6.1.2 Management and Access Control . . . 8
6.1.3 Security and Authentication . . . 8
6.1.4 Server Interfaces and Socket Bindings . . . 8
6.1.5 Deployment and Monitoring . . . 8
6.1.6 Thread and Resource Management . . . 8
6.2 Source Code Structure . . . 9
6.3 References . . . 9

1 Design Documentation (Group Work)

1.1 Entity-Relationship Diagram

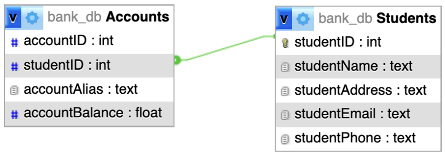

Figure 1: Entity Relationship Diagram for the Database

1.2 Architecture Diagram

`architecture_placeholder.png`

Figure 2: Overall Application Architecture

2 Screenshots (Group Work)

Add one screenshot per implemented feature, e.g., student listing, account creation, transfers.

create_account.png

Figure 3: Create Account Feature

Feature List:

This section outlines the implemented features of the Banking Web Application, categorized by functionality. Each category includes a description and a self-assessment box (1 = Poor, 5 = Excellent) to reflect how well the features were implemented.

2.0.1 Authentication & Session Management

- **Student Login:** Dropdown menu for selecting and logging in a student.
- **Session Cookies:** Username and student ID stored in cookies.
- **Logout:** Clears cookies and session state.

Implementation Rating (1–5): _____

2.0.2 Student Management (CRUD)

- Create student with form (name, email, phone, address).
- List all students (admin view).

- RESTful API: Create, Read, Update, Delete student.
- Cascade delete verified upon student deletion.

Implementation Rating (1–5): _____

2.0.3 Account Management (CRUD)

- Create account (initial balance £0, linked via student ID from cookie).
- Conditional listing (all accounts or student-specific).
- Detailed view per account with deletion.
- RESTful API: Create, Read, Update, Delete account.

Implementation Rating (1–5): _____

2.0.4 Transactions (Deposit, Withdraw, Transfer)

- Deposit and withdraw via dynamic form.
- Overdraft protection prevents excessive withdrawal.
- Transfer funds between accounts with validation.
- TransactionResult page displays updated balances and outcome messages.

Implementation Rating (1–5): _____

2.0.5 Front-End Features

- Dynamic page loading using JavaScript and AJAX.
- Button and form action binding via `main.js`.
- Confirmation prompts for sensitive actions (e.g., delete).
- Responsive feedback and clean UI structure.

Implementation Rating (1–5): _____

2.0.6 RESTful API (JSON)

- JSON-based API for students and accounts.
- Endpoints for CRUD operations.
- Special endpoints for deposit, withdraw, and transfer.

Implementation Rating (1–5): _____

Note: All JSON endpoints consume and produce data via `MediaType.APPLICATION_JSON`.

3 Evaluation (Group Work)

4 Research (Individual)

Write approx. 200 words. Compare Jakarta EE with frameworks such as:

- Spring Boot
- ASP.NET Core
- Django
- Node.js (Express)

Discuss component-based design, scalability, learning curve, IDE/tooling support, REST and ORM integration.

5 Individual Reflection (Individual)

Write approx. 200 words. Discuss your role in the group project, lessons learned, collaboration, and areas for improvement. Mention tools used (e.g., Git, Eclipse, phpMyAdmin) and key takeaways.

6 Appendix

6.1 WildFly 33 Server Configuration

This section outlines the essential configuration settings from the `standalone-full.xml` used in the deployment of the WildFly 33 application server.

6.1.1 Subsystems and Extensions

The configuration includes a comprehensive set of extensions and subsystems to support a variety of Java EE technologies:

- **EJB, JPA, JSF, JAX-RS:** Enabled via `org.jboss.as.ejb3`, `org.jboss.as.jpa`, etc.
- **Security:** Configured through WildFly Elytron with domains `ApplicationDomain` and `ManagementDomain`.
- **Clustering and Caching:** Infinispan containers set for EJBs and Web session management.
- **Logging:** Console and periodic file handlers with custom formatters.
- **Data Sources:**
  - `ExampleDS`: In-memory H2 database.
  - `BankDS`: MySQL datasource connecting to `bank_db` using driver `com.mysql.cj.jdbc.Driver`

6.1.2 Management and Access Control

- Management is accessible via HTTP interface bound to `management-http` (port 9990).
- Role-based access is enabled using the `SuperUser` role mapped to the local user `$local`.
- Audit logging is available but currently disabled.

6.1.3 Security and Authentication

- Elytron subsystems define HTTP and SASL authentication factories.
- Identity realms and properties realms are used to manage users and roles.
- A keystore `applicationKS` is defined for SSL contexts.

6.1.4 Server Interfaces and Socket Bindings

- Interfaces are defined for `management`, `public`, and `unsecure` IP addresses.
- Common ports include:
  - HTTP: 8080
  - HTTPS: 8443
  - AJP: 8009
  - Management HTTP/HTTPS: 9990 / 9993

6.1.5 Deployment and Monitoring

- Deployment scanner monitors the `deployments` directory every 5 seconds.
- Metrics, health, and microprofile extensions are enabled.

6.1.6 Thread and Resource Management

- Default thread pools are defined for concurrency and EJB processing.
- Resource adapters and managed executors are configured under the EE subsystem.

This configuration enables a robust and secure environment suitable for enterprise Java applications, with database integration, secure authentication, clustering, and deployment automation.

6.2 Source Code Structure

Briefly outline your directory structure:

```
BankApp/

  src/
     controller/
     dao/
     model/
     service/
     rest/
  webapp/
     jsp/
        accounts/
        home/
        students/
        transactions/
     css/
     js/
  META-INF/
```

6.3 References

- Jakarta EE Documentation: `https://jakarta.ee`
- WildFly Docs: `https://docs.wildfly.org`
- MySQL Documentation: `https://dev.mysql.com/doc/`